

딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사



목차

A table of Contents

01 오프닝과 딥러닝 소개

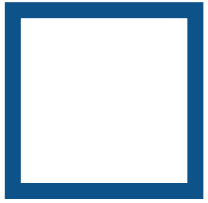
02 딥러닝 미리 보기

03 이미지는 숫자다

04 Tensor와 GPU

05 AI가 배우는 순간

06 첫 AI 만들기



PART 01

오프닝과 딥러닝 소개

1. 오프닝: AI가 사진을 맞히는 15줄

시연. 학습된 AI(ResNet-18)에 사진 넣기 (5파트에서 직접 해 봅니다)

```
from PIL import Image
import torch
from torchvision.models import resnet18, ResNet18_Weights
weights = ResNet18_Weights.DEFAULT # ImageNet으로 학습된 가중치
model = resnet18(weights=weights).eval() # 사전학습 모델
preprocess = weights.transforms() # 크기 조정, 정규화
img = Image.open("sample.jpg").convert("RGB") # 사진 (고양이, 자동차, 음식 사진 등)
x = preprocess(img).unsqueeze(0) # (1, 3, 224, 224)
print(x.shape) # → torch.Size([1, 3, 224, 224])
with torch.no_grad():
    out = model(x)
prob = out.softmax(dim=1) # 클래스 확률
top = prob.topk(3) # 상위 3개
for p, i in zip(top.values[0], top.indices[0]):
    print(f"{weights.meta['categories'][i]:20s} {p.item():.1%}")
# → (예) tabby cat          71.3%
# → (예) tiger cat         12.0%
# → (예) Egyptian cat      5.4%
```

- 지금은 결과만 보세요

- 사진 한 장 → 1000가지 사물 중 Top-3 추측과 확률

1. 오프닝 12주 로드맵

교육 단계	핵심 개념
Python 및 PyTorch 기초	Python, NumPy, Tensor, GPU
데이터 파이프라인 구성	Dataset, DataLoader, Batch, Shuffle
신경망 및 MLP 구현	Perceptron, MLP, Linear, Activation
모델 학습, 검증 및 Baseline 구축	Loss, Backpropagation, Optimizer, Train/Validation
CNN 기반 이미지 딥러닝	Convolution, Pooling, Feature Map
일반화 성능 및 데이터 개선	Overfitting, Dropout, BatchNorm, Augmentation
하이퍼파라미터 튜닝	Learning Rate, Batch Size, Optimizer, Scheduler
모델 구조 튜닝 및 최적화	Depth, Width, Filter, Activation
Transfer Learning 및 Fine-tuning	Pretrained Weight, Freeze, Unfreeze
객체탐지 모델 학습 및 튜닝	Bounding Box, IoU, Precision, Recall, mAP
실험 설계 및 결과 해석	Experiment Design, Ablation, Reproducibility
종합 프로젝트	Baseline, Tuning, Fine-tuning, Evaluation

1. 오프닝: 개발 환경 Anaconda / Google Colab

항목	실습실 PC (Anaconda + Jupyter Notebook or VScode)	Google Colab
준비	<code>conda activate dl → jupyter notebook</code>	브라우저에서 colab.research.google.com → 새 노트
GPU	실습실 PC의 NVIDIA GPU 사용	첫 셀 전에 런타임 > 런타임 유형 변경 > T4 GPU (나중에 바꾸면 올린 파일이 사라짐)
파일	노트북과 같은 폴더에 사진 넣기	왼쪽 폴더 아이콘 → 업로드
언제	수업 기본 환경	집에서 복습, 과제, 설치가 안 될 때

```
conda create -n dl python=3.11
→ pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
→ pip install numpy matplotlib jupyter
```

1. 오프닝: CUDA, cuDNN

확인 셀, 이 3줄이 나오면 GPU 준비 끝

```
!nvidia-smi # GPU와 드라이버 확인 (표가 나오면 GPU 인식 OK)
import torch
print(torch.__version__) # 2.13.0+cu126
print(torch.cuda.is_available()) # True
print(torch.cuda.get_device_name(0)) # NVIDIA RTX A4000
```

TIP Colab은 CUDA와 cuDNN이 이미 설치되어 있음, 런타임 유형만 GPU로 바꾸면 끝

1. 오프닝: 설치 확인

설치 확인, `conda activate dl` → jupyter notebook (또는 Colab)

```
import sys, torch, numpy as np
print(sys.version.split()[0]) # → (예) 3.11.9
print(torch.__version__)     # → (예) 2.13.0+cu126
print(np.__version__)         # → (예) 2.1.3
print(torch.cuda.is_available())
# → (예) True (GPU가 없으면 False)
```

TIP 오늘 자료 두 개는 첫 실행 때 자동 다운로드: 손글씨 데이터(약 10MB, 3파트), AI 가중치(약 45MB, 5파트), 실습실 PC에는 미리 받아 둬, 집에서는 과제 전에 한 번 실행해 두면 기다림 없음

- 하는 법

- 셀에 왼쪽 코드를 붙여 넣고 **Shift + Enter**, 네 줄이 나오면 준비 끝

- 출력 읽기

- `+cu126`은 GPU 빌드, 마지막 줄이 `False`여도 오늘 실습은 대부분 CPU로 가능

1. 오프닝: 노트북 사용법

- 노트북 = 코드 상자(셀)를 하나씩 실행하는 공책
 - 셀에 코드를 쓰고 **Shift + Enter** → 결과가 바로 아래에 나옴
 - 위에서부터 순서대로 실행하는 것이 기본, 꼬이면 처음부터 다시 실행 (Jupyter: Kernel > Restart & Run All, Colab: 런타임 > 모두 실행)

1. 오프닝: AI, 머신러닝, 딥러닝



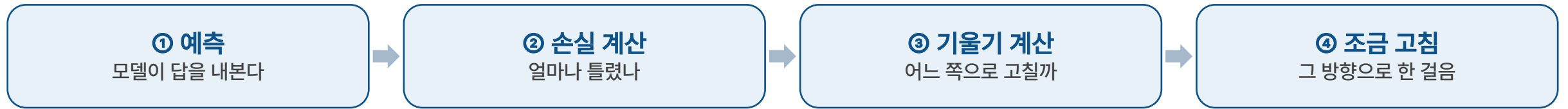
- 용어 세 개, 관계는 한 줄

- AI: 사람처럼 판단하는 소프트웨어 전부
- 머신러닝: 그중 데이터로 배우는 방법
- 딥러닝: 그중 신경망을 깊게 쌓은 방법
- 예: 규칙으로 짠 로봇청소기 경로는 AI, 스팸 필터는 머신러닝, 사진 분류/번역/챗봇은 딥러닝

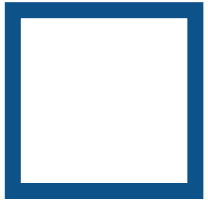
1. 오프닝: 규칙을 짜는 프로그램 vs 데이터로 배우는 프로그램

	전통 프로그래밍	머신러닝 / 딥러닝
사람이 주는 것	규칙(코드) + 입력	입력 + 정답
컴퓨터가 내는 것	출력	규칙 (= 모델)
손글씨 인식이라면	'위가 둥글고 아래가...' 규칙을 직접 코딩, 사실상 불가능	손글씨 + 정답 수만 장 → 구별 규칙을 스스로 찾음
성능 올리기	사람이 코드 수정	데이터를 더 주거나 더 오래 학습

1. 오프닝: 딥러닝 루프



- 학습 = ①→②→③→④를 수없이 반복하는 것
 - 12주 내내 이 그림, 모델과 데이터가 커질 뿐 바뀌는 것 (수식과 도구 정리는 4주차)



PART 02

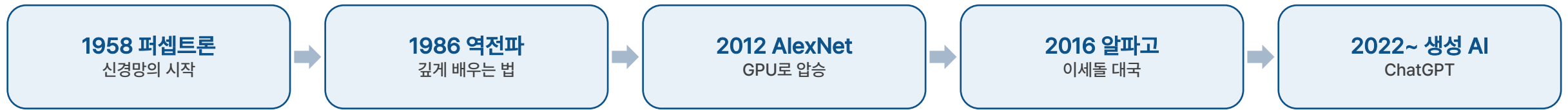
딥러닝: 이론과 수학

2. 딥러닝: 딥러닝이 하는 대표적 세 가지

무엇	예
분류 (Classification)	사진 → 무엇인지 라벨 하나 (고양이, 불량품, 숫자)
탐지 (Detection)	사진 → 무엇이 어디에 있는지 상자로
생성 (Generation)	글, 그림, 코드를 새로 만들기 (ChatGPT 등)

- 데이터에서 패턴을 스스로 학습
- 손글씨 인식이 가장 기본인 분류

2. 딥러닝: 짧은 역사, 두 번의 겨울과 부활

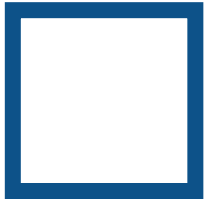


- 두 번의 겨울

- 아이디어(신경망)는 60년 전부터 있었지만, 데이터와 계산 속도 부족 등 여러 이유로 두 번의 겨울이 찾아옴

- 판을 바꾼 것

- 2012년, GPU + 대량 데이터로 학습한 AlexNet이 이미지 인식 대회를 압도 → 지금의 딥러닝 시대



PART 03

이미지는 숫자: 사진 열어 보기

3. 이미지는 숫자: sample.jpg

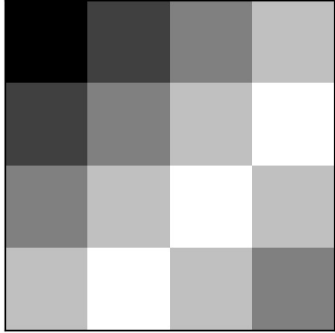
준비 셀. 내 사진(my_photo.jpg) → 320×240 → sample.jpg (없으면 샘플 생성)

```
from PIL import Image, ImageOps
import numpy as np, os
src = "사진 경로" # 사진 경로
if os.path.exists(src):
    img = ImageOps.exif_transpose(Image.open(src)).convert("RGB")
    ImageOps.fit(img, (320, 240)).save("sample.jpg")
else: # 사진이 없으면 샘플 이미지 생성
    yy, xx = np.mgrid[0:240, 0:320]
    img = np.stack([(xx / 320 * 255).astype(np.uint8),
                    (yy / 240 * 255).astype(np.uint8),
                    np.full((240, 320), 120, np.uint8)], axis=-1)
    img[(xx - 160) ** 2 + (yy - 120) ** 2 < 60 ** 2] = 255
    Image.fromarray(img).save("sample.jpg")
print(np.array(Image.open("sample.jpg")).shape) # (240, 320, 3)
```

- 딱 한 번, 실행만 하면 됩니다
 - 이 셀은 이해하지 않아도 됩니다, 내 사진을 오늘 수업 크기(320×240)로 맞추는 준비 셀
 - my_photo.jpg를 노트북 옆(Colab: 왼쪽 폴더)에 올리고 실행 → sample.jpg 생성
 - 사진이 없으면 샘플 이미지가 자동으로 만들어짐
- 마지막 줄이 (240, 320, 3)이면 성공

3. 이미지는 숫자: 사진 = 큰 숫자 표

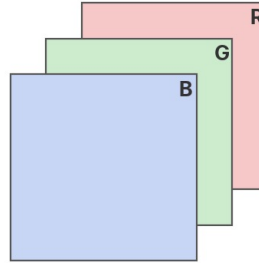
흑백 이미지 (4×4 픽셀)



같은 이미지의 숫자 배열
shape (4, 4), dtype uint8

0	64	128	192
64	128	192	255
128	192	255	192
192	255	192	128

컬러 이미지의 채널



컬러 이미지 = 채널 3장 → shape (H, W, 3)
값 범위 0~255 → /255 로 0~1 정규화

- 사진의 정체

- 흑백은 점(픽셀) 하나 = 0~255 숫자 하나 (0 = 검정, 255 = 최대 밝기), 사진 = 그 숫자를 모아 놓은 큰 표
- 컬러는 빨강, 초록, 파랑 표 3장이 겹친 것, 그래서 컬러 픽셀 하나는 (R, G, B) 숫자 3개

- shape 읽는 법

- (240, 320, 3) = 세로 240줄 × 가로 320칸 × 색 3장
- 사진 한 줄 = 벡터, 흑백 사진 = 행렬

3. 이미지는 숫자: 사진 편집 명령 4개

하고 싶은 것	명령 한 줄	읽는 법
자르기	<code>a[60:180, 100:220]</code>	세로 60~179줄, 가로 100~219칸만 남김
뒤집기 (좌우)	<code>a[:, ::-1]</code>	<code>::-1</code> = 칸 순서를 거꾸로
어둡게	<code>(a * 0.5).astype(np.uint8)</code>	모든 픽셀에 $\times 0.5$ 를 한 번에
흑백	<code>a.mean(axis=2).astype(np.uint8)</code>	R, G, B의 평균

- `:` 하나만 쓰면 전부 (예: `a[:, ::-1]` = 모든 줄에서 칸 뒤집기), `[:120]`은 처음부터 120 앞까지
- 곱하기 0.5 = 어둡게, 255에서 빼기 = 반전, 255로 나누기 = 0~1로 (4파트 레시피의 `/255`)
- `astype(np.uint8)`은 계산 결과(소수)를 이미지용 정수(0~255)로 되돌리는 마무리, 세트로 붙인다고만 기억

3. 이미지는 숫자다: 사진을 열어 숫자 보기

예제 3-1. 사진 → 숫자 표 → 픽셀 찍어 보기

```
from PIL import Image
import numpy as np
img = Image.open("sample.jpg")
a = np.array(img) # 사진 → 숫자 배열
print(a.shape, a.dtype) # (240, 320, 3) uint8
print(a[0, 0]) # [0 0 119] (왼쪽 위 픽셀의 R, G, B)
print(a[120, 160]) # [255 255 255] (가운데 픽셀)
print(a.min(), a.max()) # 0 255 (다른 값일 수 있음)
print(a[:, :, 0].mean().round(1)) # 145.3(R 평균, 다른 값)
```

- 두 줄이면 사진이 숫자가 됩니다
 - `Image.open`으로 열고 `np.array`에 넣으면 끝
 - `a.shape = (240, 320, 3)`, `a[0, 0] =` 왼쪽 위 픽셀의 (R, G, B)
- 오늘의 핵심 문장
 - 사진은 그냥 큰 숫자 표다, 그래서 계산으로 편 집할 수 있다

3. 이미지는 숫자다: 사진 편집 4종

예제 3-2. 자르기, 뒤집기, 어둡게, 흑백, 명령 4개 그대로

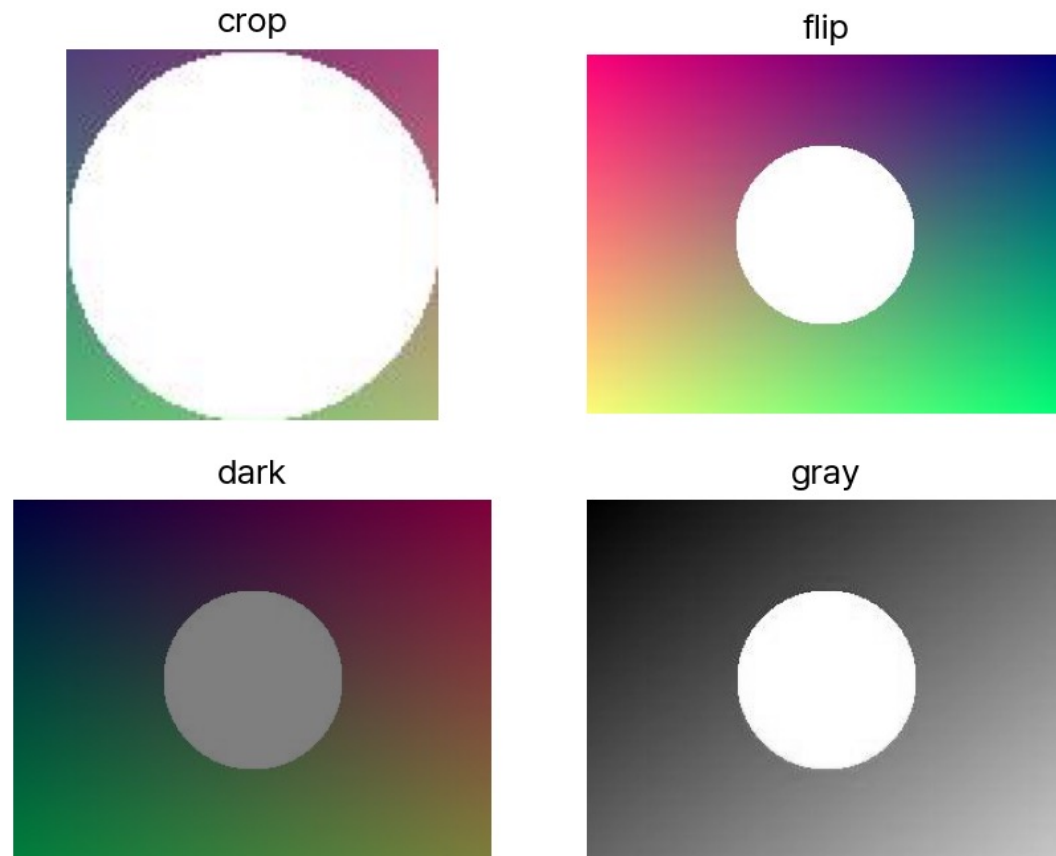
```
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

a = np.array(Image.open("sample.jpg")) # (240, 320, 3)
crop = a[60:180, 100:220] # 자르기: 행 60~179, 열 100~219
flip = a[:, ::-1] # 좌우 반전: 열 순서 뒤집기
dark = (a * 0.5).astype(np.uint8) # 어둡게: 모든 픽셀 × 0.5
gray = a.mean(axis=2).astype(np.uint8) # 흑백: R,G,B 평균
pairs = [("crop", crop), ("flip", flip), ("dark", dark), ("gray", gray)]
for i, (name, im) in enumerate(pairs):
    plt.subplot(2, 2, i + 1); plt.imshow(im, cmap="gray"); plt.title(name);
plt.axis("off")
plt.show()
print(crop.shape, flip.shape, gray.shape) # (120, 120, 3) (240, 320, 3) (240, 320)
```

- 위 네 줄이 전부

- 앞 장 표의 명령 4개를 그대로 친 것
- 아래 `for`와 `plt.` 줄들은 네 장을 나란히 보여 주는 표시용, 이해하지 않아도 됨

3. 이미지는 숫자: 결과



- 한 줄 = 효과 하나
 - 자르기 = 범위 고르기, 반전 = `::-1`, 어둡게 = $\times 0.5$, 흑백 = 세 색 평균
- 이게 딥러닝에서 쓰이는 곳
 - 학습 사진을 뒤집고 잘라 데이터를 늘리는 기법

3. 이미지는 숫자다 (6) 손글씨 숫자(MNIST)를 글자로 찍어 보기

예제 3-3. 손글씨 한 장 → 숫자 표 → 글자로 찍기

```
import matplotlib.pyplot as plt
import torch
from torchvision import datasets
mnist = datasets.MNIST(root="data", train=True, download=True) # 첫 실행 시 다운로드
img, label = mnist[0] # 0번째 샘플: (PIL 이미지, 정답)
t = torch.tensor(list(img.getdata())).view(28, 28) # 픽셀 → (28, 28)
print(t.shape, t.max().item(), label) # → torch.Size([28, 28]) 255 5
for row in t[6:22:3]: # ASCII 아트 (3줄마다)
    print("".join("#" if v > 128 else "." for v in row))
plt.imshow(t.numpy(), cmap="gray"); plt.title(f"label = {label}"); plt.show()
```

- 숫자 표를 글자로 찍으면

- 값이 크면 #, 작으면 . → 화면에 숫자 5가 나타남
- 정답(label)도 5, 그림과 정답이 쌍으로 들어 있는 데이터

3. 이미지는 숫자: 실습 1 사진 편집 (문제)

실습

1. 사진 배열 a

① 상하 반전

② 왼쪽 위 1/4만 남기기

③ 이어서 1.5배 밝게(255를 넘지 않게)

④ 흑백

각 결과를 plt.imshow로 확인

• 힌트

- ① 줄 순서 뒤집기 `a[::-1]` ② 줄 0~119, 칸 0~159 ③ `np.clip(a * 1.5, 0, 255).astype(np.uint8)` ④ `a.mean(axis=2).astype(np.uint8)`
- 그리는 법: `plt.imshow(결과, cmap="gray")` 그리고 `plt.show()` (흑백은 cmap 필요)

• 확인

- ①은 위아래가 뒤집힌 내 사진, ②는 shape (120, 160, 3), ④는 shape (240, 320)

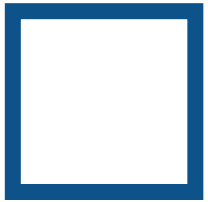
3. 이미지는 숫자다 (8) 실습 1: 정답 예시

실습 1. 정답 예시

```
from PIL import Image
import numpy as np
a = np.array(Image.open("sample.jpg"))
updown = a[::-1] # ① 상하 반전
quarter = a[:120, :160] # ② 왼쪽 위 1/4
bright = np.clip(a * 1.5, 0, 255).astype(np.uint8)
# ③ 1.5배 밝게
gray = a.mean(axis=2).astype(np.uint8)
# ④ 흑백
print(updown.shape, quarter.shape)
# → (240, 320, 3) (120, 160, 3)
print(bright.max(), gray.shape)
# → 255 (240, 320)
```

- 포인트

- 전부 명령 4개 표에서 숫자만 바꾼 것
- `np.clip(..., 0, 255)`: 255를 넘는 값 자르기, 안 하면 값이 넘쳐 이상한 색이 됨



PART 04

Tensor와 GPU

4. Tensor와 GPU : Tensor = GPU로 가는 배열

0차원: 스칼라
shape = ()

7

1차원: 벡터
shape = (4,)

1	2	3	4
---	---	---	---

2차원: 행렬
shape = (3, 4)

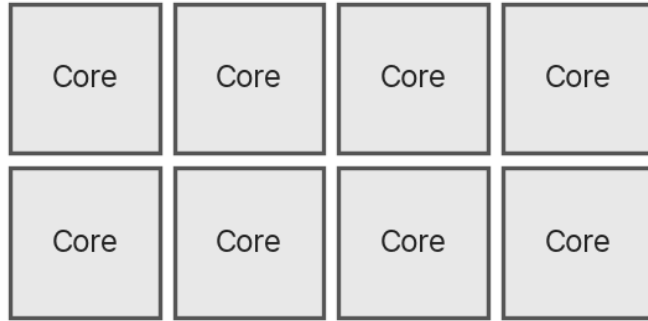
0	1	2	3
4	5	6	7
8	9	10	11

3차원: Tensor
shape = (2, 3, 4)

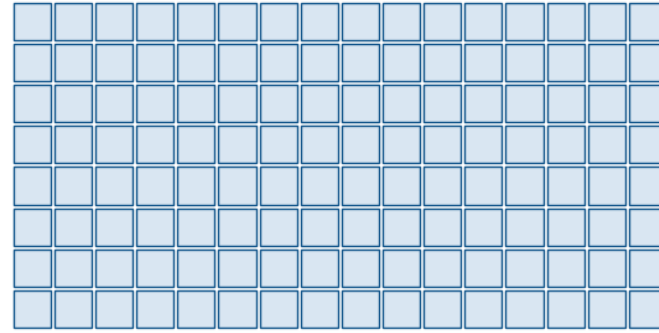
0	1	2	3
4	5	6	7
8	9	10	11

- Tensor 특징 2가지
 - ① GPU에서 계산 가능, ② 자동 미분
- shape
 - (240, 320, 3) 사진 한 장, (4, 3, 240, 320) 사진 4장 묶음 (맨 앞 = 장수)

4. Tensor와 GPU: 왜 GPU인가



CPU: 소수의 강력한 코어 (4~32개)
복잡한 순차 작업, 제어에 강함



GPU: 수천 개의 단순 코어 (예: 수천~수만 개)
동일한 연산을 대량 데이터에 동시에 적용 (행렬곱)

- 비유 하나

- 수학 문제 100만 개를 풀 때, 교수 8명(CPU)보다 학생 5천 명(GPU)이 빠르다
- 딥러닝 계산 = 서로 독립인 곱셈 수백만 개 → 동시에 하면 십수~수백 배

- 조건 하나

- 데이터를 GPU 메모리(VRAM, Video RAM)로 옮겨 줘야 계산 가능

4. Tensor와 GPU: NumPy ↔ Tensor

예제 4-1. NumPy → Tensor → 다시 NumPy

```
import numpy as np, torch
a = np.array([[1, 2, 3], [4, 5, 6]])
# NumPy 배열 (사진과 같은 종류)
t = torch.from_numpy(a)
# Tensor로 바꾸기: 한 줄
print(t.shape)
# → torch.Size([2, 3])
print(t * 2)
# tensor([[ 2,  4,  6],
#         [ 8, 10, 12]])
print(t.float().mean()) # tensor(3.5000)
print(t.numpy().shape)  # (2, 3)
```

4. Tensor와 GPU: CPU vs GPU 속도 1

예제 4-3. 행렬 계산 시간 재기

```
import torch, time
def bench(device, n=1024, repeat=3):
    a = torch.randn(n, n, device=device)
    b = torch.randn(n, n, device=device)
    c = a @ b # 위밍업
    if device.type == "cuda": torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(repeat):
        c = a @ b
    if device.type == "cuda": torch.cuda.synchronize()
    return (time.perf_counter() - t0) / repeat * 1000

print(f"CPU : {bench(torch.device('cpu')):.2f} ms")
# → (예) CPU : 4.57 ms
if torch.cuda.is_available():
    print(f"GPU : {bench(torch.device('cuda')):.2f} ms")
# → (예) GPU : 0.16 ms
```

- 포인트

- bench는 시간을 재 주는 도구
- 바꿀 곳은 n= 숫자 하나

- 결과는 하드웨어마다 다름

4. Tensor와 GPU: CPU vs GPU 속도 2

예제 4-4. n별 시간 자동 출력 (bench는 예제 4-3 그대로)

```
import torch, time
# bench(device, n, repeat=3)는 함수를 그대로 사용
devs = [torch.device("cpu")]
if torch.cuda.is_available():
    devs.append(torch.device("cuda"))
for n in (256, 1024, 4096):
    print(n, " | ".join(f"{bench(d, n):7.2f} ms" for d in devs))
```

→ (예) 256 0.15 ms | 0.04 ms
→ (예) 1024 4.57 ms | 0.16 ms
→ (예) 4096 99.05 ms | 6.58 ms

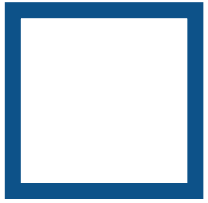


PART 05

AI가 배우는 순간: 직선 맞추기

AI가 배우는 순간: 직선 맞추기

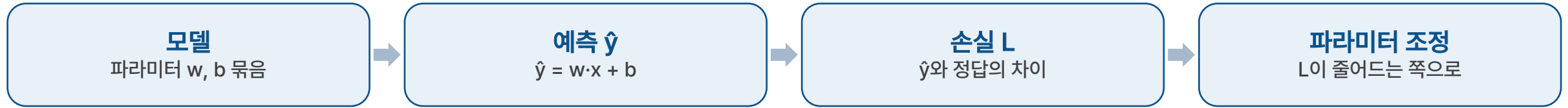
- 이론: 학습의 정의, 손실 함수(MSE), 경사하강법, 신경망, 오늘의 수식 2개
- 컴퓨터가 스스로 직선을 찾아내는 장면을 직접 본다
- 실습 3: 학습률 실험
- 마무리 체험: 내 사진을 AI가 분류한다



PART 05

AI가 배우는 순간: 직선 맞추기

5. AI가 배우는 순간: 학습 = 손실을 줄이는 일



- 용어 세 개

- 파라미터: 모델 안의 조절 손잡이 (직선은 w , b 2개)
- 손실(loss): 지금 얼마나 틀렸는지를 숫자 하나로, 작을수록 좋다
- 학습: 손실이 줄어드는 방향으로 손잡이를 조금씩 돌리는 반복

5. AI가 배우는 순간: 손실 함수

손실 함수 MSE, 수식을 코드 두 줄로

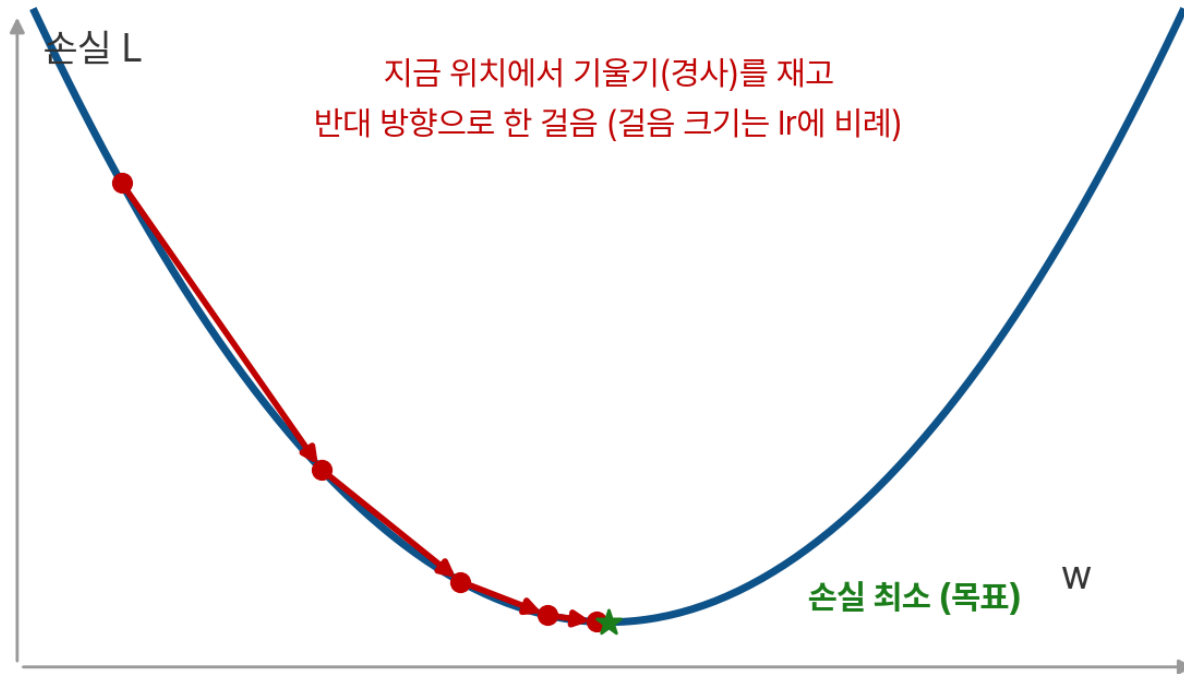
```
import torch
y_pred = torch.tensor([2.5, 0.0, 2.0])
# 모델의 예측
y_true = torch.tensor([3.0, -0.5, 2.0])
# 정답
err = y_pred - y_true
# 오차
print(err)
# → tensor([-0.5000,  0.5000,  0.0000])
loss = (err ** 2).mean()
# 제공해서 평균 = MSE
print(loss)
# → tensor(0.1667)
```

- 틀린 정도를 재는 자 (회귀용 MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- 말로 하면: (예측 - 정답)을 제공해서 평균
- 제공하는 이유: 부호를 없애고, 크게 틀린 것에 더 큰 벌점
- 왼쪽 코드 두 줄이 이 수식의 전부

5. AI가 배우는 순간: 경사하강법



- 비유: 안개 낀 산 내려오기

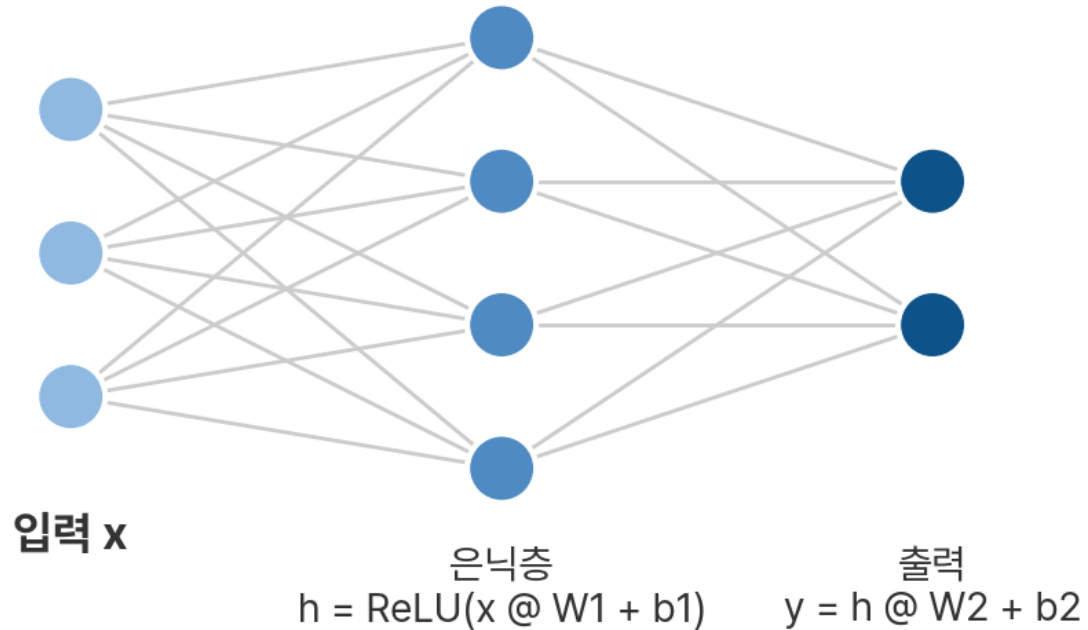
- 멀리는 안 보여도 발밑의 경사는 안다 → 낮은 쪽으로 한 걸음, 반복하면 골짜기

$$w \leftarrow w - lr \times \frac{\partial L}{\partial w}$$

- $\frac{\partial L}{\partial w}$ (기울기)는 PyTorch `backward()`가 자동 계산, 미분을 손으로 할 일 없음
- `lr`(학습률) = 걸음 크기, 너무 크면 튕겨 나감, 너무 작으면 느림

5. AI가 배우는 순간: 신경망

층 하나 = 행렬곱 @ 한 번 (2파트), ReLU: 음수는 0으로

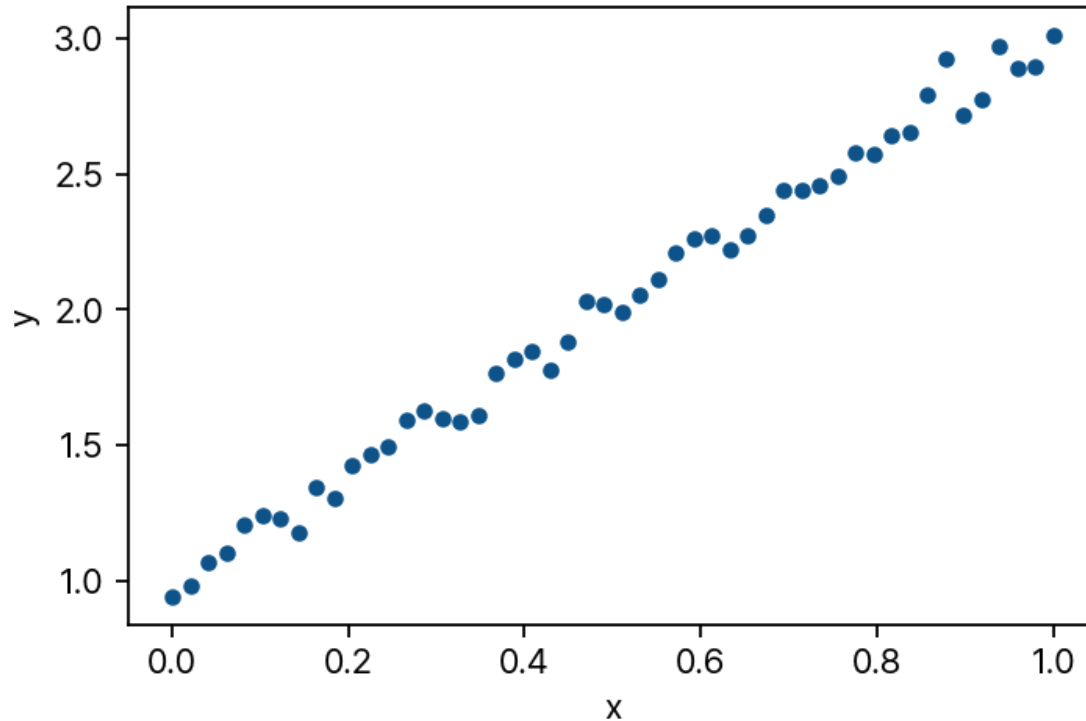


- 신경망의 정체

- 층 = 숫자들을 섞어 새 숫자들을 만드는 계산 한 번 = 행렬곱 @, 그걸 겹겹이 쌓은 것이 신경망
- 층 사이의 ReLU = 음수를 0으로 바꾸는 아주 단순한 장치
- 직선 $y = w \cdot x + b$ 는 가장 작은
- deep learning = 층을 깊게 쌓았다는 뜻

5. AI가 배우는 순간

점 50개, 이 점들을 가장 잘 지나는 직선 $y = w \cdot x + b$ 의 w , b 는? • 상황



• 상황

- 점 50개가 있고, 이 점들을 가장 잘 지나는 직선의 w 와 b 를 모른다
- 사람은 눈으로 굿지만, 컴퓨터에게는 '틀린 만큼 고쳐라' 규칙만 준다

• 이것이 학습

- 직선 대신 신경망, 점 50개 대신 사진 6만 장이 되어도 똑같은 규칙으로 배운다

5. AI가 배우는 순간

예제 5-1. backward() 한 줄이 '고칠 방향'을 계산

```
import torch
x = torch.tensor(3.0, requires_grad=True) # 추적
y = x ** 2 + 2 * x                        #  $y = x^2 + 2x$ 
y.backward()                             #  $dy/dx$  자동 계산
print(y.item(), x.grad)                  # → 15.0 tensor(8.)

w = torch.tensor([1.0, -1.0], requires_grad=True)
loss = (w * torch.tensor([2.0, 3.0])).sum()
loss.backward()
print(w.grad)                            # → tensor([2., 3.])
with torch.no_grad():                    # 추적 없이 계산
    w -= 0.1 * w.grad                    # 경사하강 한 걸음
print(w.detach())                        # → tensor([ 0.8000, -1.3000])
```

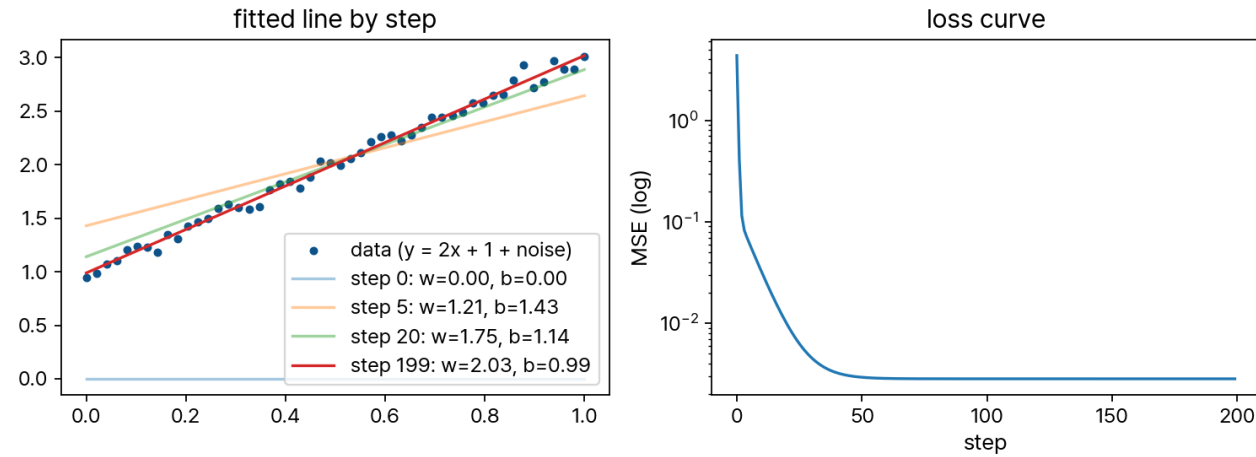
5. AI가 배우는 순간

예제 5-2. $y = 2x + 1$ 을 모르는 상태에서 w, b 찾기

```
import torch
torch.manual_seed(0)
x = torch.linspace(0, 1, 50)
y = 2 * x + 1 + 0.05 * torch.randn(50)
# 정답: 기울기 2, 절편 1 (+ b)
w = torch.tensor(0., requires_grad=True)
# 학습할 파라미터 (처음엔 0)
b = torch.tensor(0., requires_grad=True)
for step in range(200):
    loss = ((w * x + b - y) ** 2).mean() # 예측과 정답의 차이 (MSE)
    loss.backward() # dloss/dw, dloss/db 자동 계산
    with torch.no_grad():
        w -= 0.5 * w.grad; b -= 0.5 * b.grad # 기울기 반대 방향으로 한 걸음
        w.grad.zero_(); b.grad.zero_() # 기울기 초기화 (누적 방지)
print(round(loss.item(), 4)) # → 0.0028
print(round(w.item(), 2), round(b.item(), 2))
# → 2.03 0.99 (정답 2, 1에 근접!)
```

- 루프 안 = 네 단계 바퀴 그대로
 - 예측 → 손실(MSE) → `backward()` → 한 걸음
- 실행 결과
 - 200번 반복 → $w = 2.03$, $b = 0.99$
 - 정답(2와 1)을 모델에게 알려준 적이 없는데 점 데이터만 보고 찾아냈다, 이것이 학습

5. AI가 배우는 순간: 결과



- 무슨 일이 일어났나

- step 0: 평평한 선(손실 큼) → step 5, 20: 방향을 잡아감 → step 199: 정답 도착
- 오른쪽 손실 곡선이 뚝 떨어지는 것 = 배우고 있다는 증거

5. AI가 배우는 순간: 실습 3

실습

3. ① 정답을 $y = -3x + 0.5$ + 잡음으로 바꾸고 300번 학습 → w, b 출력
② 학습률(0.5라고 쓰인 두 곳)을 2.0으로, 다시 0.01로 바꿔서 정답을 최대한 근접하게 맞춰보기

5. AI가 배우는 순간: 실습 3

실습 3.

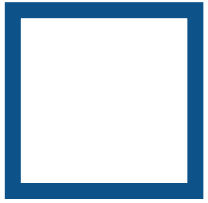
```
import torch
torch.manual_seed(0)
x = torch.linspace(0, 1, 50)
y = -3 * x + 0.5 + 0.05 * torch.randn(50) # 정답: 기울기 -3, 절편 0.5
w = torch.tensor(0., requires_grad=True)
b = torch.tensor(0., requires_grad=True)
for step in range(300): # Q1: 학습률 0.5로 300번
    loss = ((w * x + b - y) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        w -= 0.5 * w.grad; b -= 0.5 * b.grad
        w.grad.zero_(); b.grad.zero_()
print(round(w.item(), 2), round(b.item(), 2)) # → -2.97 0.49
# Q2: 학습률을 2.0으로 바꾸면? (발산) / 0.01로 바꾸면? (느린 수렴)
```

- 학습률
 - 2.0: 튕겨 나가 nan (발산)
 - 0.5: 300번이면 충분
 - 0.01: 같은 횟수로 도착 못 함
- 그럼 적당한 학습률은?
 - 문제마다 다름

5. AI가 배우는 순간



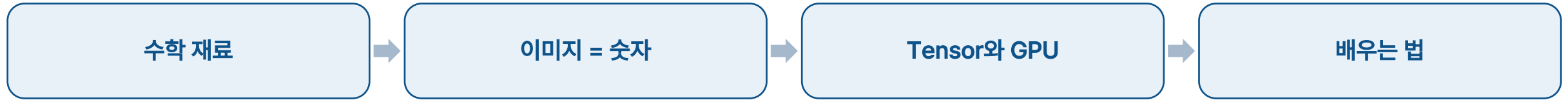
- 실습 = 이 그림의 전부
 - 사진 → 숫자 → Tensor, GPU → 모델과 학습
 - 이걸 전부 합쳐서 진짜 AI를 만듭니다



PART 06

첫 AI 만들기: 손글씨 숫자 인식

6. 첫 AI 만들기



- 네 가지 = AI를 만드는 재료 전부

- 지금부터 손글씨 숫자 인식 AI를 직접 만든다, 새로 외울 것 없음, 낯선 명령 몇 개는 주문처럼 복사, 원리는 전부 배운 것
- MNIST 손글씨 6,000장으로 훈련(Train) → 처음 보는 1,000장으로 시험(Test)

- 진행

- 준비 → 훈련 → 시험 → 틀린 문제 구경, 훈련은 GPU에서 약 1초

6. 첫 AI 만들기: 데이터 준비

예제 6-1. 훈련 데이터 준비

```
import torch
from torchvision import datasets
device = "cuda" if torch.cuda.is_available() else "cpu"
mnist = datasets.MNIST(root="data", train=True, download=True)
x = mnist.data[:6000].float().view(6000, 784) / 255
# 사진을 한 줄 784칸으로 펴기
y = mnist.targets[:6000]
# 정답 숫자 6000개
x, y = x.to(device), y.to(device)
# 둘 다 GPU로
print(x.shape, y.shape)
# → torch.Size([6000, 784]) torch.Size([6000])
print(y[:8].tolist())
# → [5, 0, 4, 1, 9, 2, 1, 3] (첫 숫자가 3파트에서 본 5)
```

- AI에게 주는 것

- x = 문제지 6,000장, y = 답안지 6,000개
- 문제와 답을 같이 줘야 배운다

6. 첫 AI 만들기: 훈련

예제 6-2. 훈련, 5파트의 루프 그대로

```
import torch.nn as nn
torch.manual_seed(0)
model = nn.Sequential(nn.Linear(784, 100), nn.ReLU(),
                      nn.Linear(100, 10)).to(device)

# 2층 신경망
opt = torch.optim.SGD(model.parameters(), lr=1.0)
loss_fn = nn.CrossEntropyLoss()
for epoch in range(301):
    loss = loss_fn(model(x), y)           # 예측 → 손실
    opt.zero_grad(); loss.backward()      # 고칠 방향 계산
    opt.step()                            # 한 걸음
    if epoch % 100 == 0:
        print(epoch, round(loss.item(), 3))
# → 0 2.303   (시작: 열 중 하나 찍기 수준)
# ex. 100 0.168
# ex. 200 0.084
# ex. 300 0.047   (300걸음 만에 약 1/50, 하드웨어마다 조금 다름)
```

- 직선 맞추기와 완전히 같은 루프

- 예측 → 손실 → `backward()` → 한 걸음, 손잡이가 `w`, `b` 2개에서 약 8만 개로 늘었을 뿐
- 시작 손실 2.303 = 열 중 하나 찍기 수준 → 300 스텝 뒤 0.05
- GPU에서 300스텝이 약 1초 (4파트
- `lr=1.0`? 학습률은 문제마다 다름, 이 문제엔 1.0이 적당
- 숫자를 구별하는 규칙은 한 줄도 쓰지 않았다, 문제와 답만 줬는데 스스로 배웠다

6. 첫 AI 만들기: 처음 보는 1,000장



- 약 920 / 1,000 정답, 3시간 전엔 `print`를 처음 치던 우리가 만든 AI
- `argmax`: 10개 점수 중 최고점 고르기 = AI의 답

예제 6-3. 시험 보기, 처음 보는 문제 1,000개

```
test = datasets.MNIST(root="data", train=False, download=True)
# 처음 보는 문제
xt = test.data[:1000].float().view(-1, 784).to(device) / 255
yt = test.targets[:1000].to(device)
pred = model(xt).argmax(dim=1)
# 10개 점수 중 최고점 = 예측
print(pred[:8].tolist())
# ex. [7, 2, 1, 0, 4, 1, 4, 9] (예측)
print(yt[:8].tolist())
# → [7, 2, 1, 0, 4, 1, 4, 9] (정답, 전부 일치)
acc = (pred == yt).float().mean().item()
print(f"정확도: {acc*100:.1f}%")
# ex. 정확도: 92.0%
```

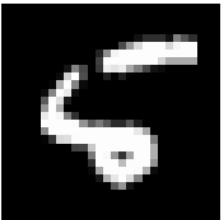
6. 첫 AI 만들기: AI가 틀린 문제

예제 6-4. 틀린 문제 구경

```
import matplotlib.pyplot as plt
wrong = (pred != yt).nonzero().view(-1) # 틀린 문제 번호들
print(len(wrong))
# ex. 80 (1000문제 중 80개)
fig, axes = plt.subplots(1, 5, figsize=(8, 2))
for ax, i in zip(axes, wrong[:5].tolist()):
    ax.imshow(test.data[i].numpy(), cmap="gray"); ax.axis("off")
    ax.set_title(f"pred {pred[i].item()} / ans {yt[i].item()}",
    fontsize=10)
plt.show()
```

- 1,000문제 중 약 80개는 틀린다
 - 틀린 문제를 직접 보면, 사람도 헛갈리는 글씨가 많음
- 앞으로 11주 = 이 92%를 끌어올리는 여정
 - 데이터 공급, 신경망 만들기, CNN → 99%+

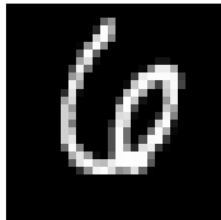
pred 6 / ans 5



pred 6 / ans 4



pred 2 / ans 6



pred 7 / ans 2



pred 4 / ans 9

